

CODE SPORT



In this month's column, we continue our discussion on information retrieval.

Last month, I focused on the basic problem of information retrieval, namely, 'ad-hoc information retrieval' where, given a collection of documents, the user can arbitrarily query for any information which may or may not be contained in the given document collection. One of the well-known examples of ad-hoc information retrieval systems is the Web search engine, using which the user can pose a query on an arbitrary topic, and the set of documents is all the searchable Web pages available on the 'World Wide Web'.

A related but different kind of information retrieval is what is known as a 'filtering' task. In the case of filtering, a user profile is created, and a set of documents that may be of relevance or interest to the user are filtered from the document collection and presented to the user. A well-known example of information filtering is the personalisation of news delivery based on the user's profile. Typically, in the case of the information filtering task, the set of documents in the document collection keeps changing dynamically (for instance, new news stories arrive and old news stories are retired from the collection), while the user query (for finding documents which are of interest based on the user profile) remains relatively stable. This is in contrast to 'ad-hoc information retrieval', wherein the document collection remains stable but queries keep changing dynamically.

In last month's column, we discussed the Boolean model of ad-hoc information retrieval, wherein user information needs are presented in terms of a keyword-based query, and the documents in the collection are looked up using a Boolean model for the presence of the keywords in the query. For example, given that the document collection is the entire works of Shakespeare, the query term 'Julius Caesar' would find the documents that contain the query terms 'Julius' and 'Caesar' in them. In order to facilitate the queries, a data structure known as inverted index is generally built. The inverted index consists of the dictionary of terms in the document collection and a 'pos list' for each term, listing the document IDs which contain that term. Typically, the 'pos list' has not just the document IDs of the documents containing the term, but also contains the 'position' in the document where

the term appears. Given the set of documents in the document collection, the pre-processing step is to construct the inverted index. So how do we construct this index?

As I mentioned before, each document is scanned and broken into tokens or terms. There are two ways of constructing an inverted index. The first one is known as the 'Blocked Sort Based Indexing' (BSBI) method. Till now, when building inverted indexes was discussed, we made an implicit assumption that the data can fit in the main memory. But that assumption does not hold true for large document collections and, obviously, for a Web search. Hence, we need to consider the case when the document collection cannot be analysed in its entirety in the main memory to construct the inverted index. That's where BSBI comes in.

BSBI divides the document collection into blocks, where each block can fit in the main memory. It then analyses one block, gets the terms in that block, and creates a sorted list of (term ID, doc ID) pairs. We can think of the sorting as being done with the 'term ID' as the primary key and the 'doc ID' as the secondary key. Note that since the block can fit in main memory, the sorting can be done in the main memory for that block and the results of the sorting are written back to the stable storage. Once all the blocks are processed, we have an 'inverted index' corresponding to each block of the document collection on disk in intermediate files. These intermediate 'inverted indices' need to be merged in order to construct the final 'inverted index' for the document collection. Let us assume that we have divided the document collection into 10 blocks. After the intermediate index construction step, we have 10 intermediate index files on disk. Now, in the final merging process, we open the 10 files, read data from each file into a read buffer, and we open a write buffer for writing back the results. During each step, we select the smallest term ID among those present in the read buffer and process it. All the document IDs corresponding to the term ID are processed and added to the 'pos list' for that term, and written back to disk. We can view this process as the equivalent of merging 10 individual sorted lists into one final sorted list. I leave it as an exercise for the reader to compute the complexity associated with BSBI.